

NeurIPS 2026 Education Track

Speculative Decoding

Making autoregressive decoding faster, without sacrificing on token quality.

Ayush Vikram, Grace Gong

Standard Autoregressive Decoding

- Generate one token, feed it back in as new input, and repeat, one token at a time.



- Each LLM call above is one pass. Passes run strictly one after another: about 200 ms + 200 ms = ~400 ms for 2 tokens (illustrative timing, not a measurement of these specific runs).
- Each pass reloads the entire model's weights just to produce one token, making decode memory-bound. Because each output token depends on the last, passes cannot run in parallel, so latency grows with every extra token.

Parallelism Won't Solve This: The Decode Bottleneck

- Because each output token depends on the ones before it, decoding a sequence cannot be parallelized across tokens, so the GPU sits underutilized.
- Generating a single token still requires loading every model weight from memory. That makes the arithmetic intensity of decode very low, far below what keeps a GPU busy.
- Can we generate multiple tokens for the same cost of loading the weights once?
- Decode: one token per step. The GPU waits on memory.
- Prefill: all prompt tokens processed at once. The GPU stays busy.

Phase 1: Draft a Sequence of Candidate Tokens

- Objective: generate a sequence of candidate tokens (“predictions”) as fast as possible.
- Method: use a secondary, smaller model, the draft model, to propose that sequence of candidate tokens.
- Key insight: a smaller model reads and writes far fewer bytes per step, so its decode will be faster.

Phase 2: Verify with One Parallel Pass

- What happens: the target model processes the input plus every drafted token in one forward pass, producing a probability distribution for each position.
- The difference from checking tokens one at a time: verifying sequentially would mean running the target model once per drafted token, exactly as slow as normal decoding. Verifying in parallel feeds the whole drafted sequence through the target in a single forward pass instead, the same trick prefill uses, so all the probability distributions come out of one pass rather than many.
- Key insight: one forward pass, at roughly the cost of generating a single token the old way, lets us verify, and potentially accept, several new tokens at once. The GPU was already underused during decode, and now it earns its keep.

Phase 3: Accept or Reject, via Rejection Sampling

- During verification, the target model decides whether to keep each drafted token based on both models' probabilities:

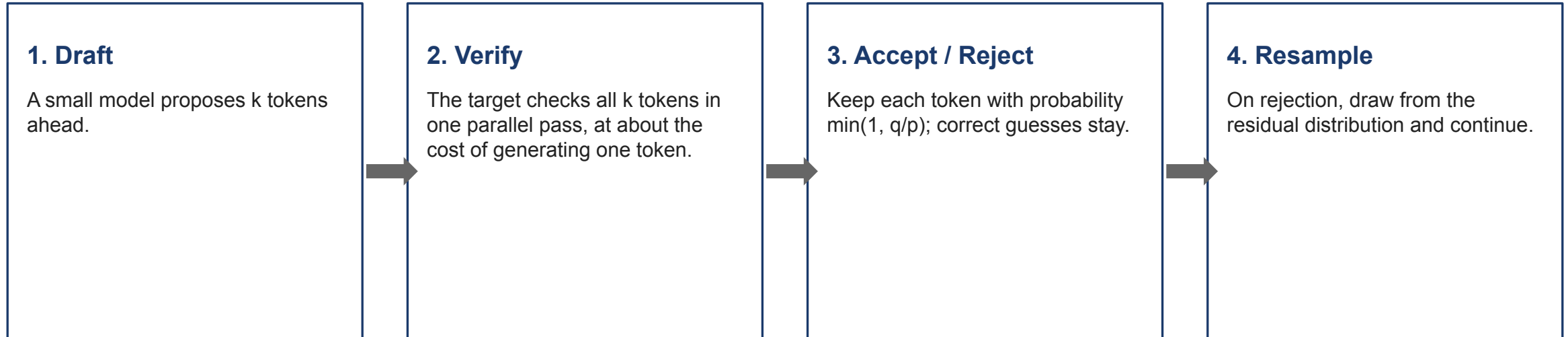
Token	Paris	is	a	historic	city
P(target) q	0.8	0.9	0.6	0.3	
P(draft) p	0.7	0.7	0.4	0.5	
Decision	Accepted	Accepted	Accepted	Rejected	never reached

- Rule: accept a drafted token with probability $\min(1, q(x) / p(x))$. If it's rejected, every later drafted token in that round is discarded too, since they were generated assuming the rejected token was real.
- Why rejection sampling specifically: it's a standard statistical method for drawing exact samples from an expensive distribution (q) using only cheap samples from an approximate one (p), by accepting or resampling based on their probability ratio. It needs no retraining and no assumptions about how p and q relate to each other, which is exactly why it carries over from statistics into speculative decoding unchanged.
- Why this is exact, not approximate: if $p(x) \leq q(x)$, accept outright, since the target is at least as confident. If $p(x) > q(x)$, accept with probability $q(x)/p(x)$; the algebra $p(x) \cdot q(x)/p(x) = q(x)$ is exactly what keeps the output distribution unbiased.

Phase 4: Handling a Rejection: Residual Resampling

- On rejection: sample a correction token from the residual distribution, $\text{normalize}(\max(0, q - p))$. This is exactly the part of the target's distribution the draft under-represented.
- After that: every later prediction in that drafted sequence is thrown away, and standard autoregressive decoding resumes from the corrected token.

The Speculative Decode Loop



A Concrete Walkthrough

- Prompt: “The capital of France is.” The draft proposes four tokens: Paris, a, beautiful, city.
- The target verifies all four in one pass. It agrees on Paris and a, but favors historic over beautiful.
- That guess is discarded and historic is sampled from the residual distribution instead. Every later prediction in that sequence goes with it.

Paris (accepted) → **a** (accepted) → **beautiful** (rejected) → resample → **historic** (kept)

city was never reached: once beautiful was rejected, the rest of that drafted sequence was discarded.

Compute, Not Compress

- Speculative decoding shifts work between models rather than shrinking them. A lighter draft model proposes tokens; the target verifies them and preserves its own exact output distribution: nothing is approximated.
- Distillation trains one faster model, but it can lose capabilities the original had.
- KV-cache reuse speeds up repeated contexts, but proposes no new tokens.
- Standard sampling calls the target model once per output token, always.
- Speculative decoding calls the target model once per accepted block, not per token.

One Model, or Two?

- Legacy (draft + target): two separate models, a small draft and a large target, kept aligned by training or by sharing a tokenizer family.
- Medusa (target + extra heads): one model, no separate draft. Extra decoding heads predict further-ahead tokens; the target verifies its own heads in one pass.
- A middle path, EAGLE and EAGLE-2: a lightweight draft head reuses the target model's own hidden states instead of running a fully separate model. EAGLE-2 goes further, proposing a small tree of candidate continuations and verifying the whole tree in one target pass with a tree-attention mask. This is the direct model for the design exercise later in the notebook.

The Acceptance Rate α

- α , the average per-token acceptance rate: each drafted token is kept with probability $\min(1, q(x)/p(x))$, and α is the expected value of that acceptance probability across tokens, where p is the draft's distribution and q the target's.
- High α , a well-aligned draft: when the draft model's distribution p tracks the target's distribution q closely, most drafted tokens survive. Many tokens get through per round, and the speedup is large.
- Low α , a poorly-aligned draft: correctness is never at risk, and the output distribution stays exact, but below a break-even acceptance rate, the cost of drafting outweighs what little gets accepted: speculative decoding ends up slower than plain autoregressive decoding.

Diminishing Returns at Larger Speculation Lengths

Speculation length k	Expected accepted tokens before first rejection ($\alpha = 0.8$)
2	2.44
4	3.36
8	4.33 (plateauing)

- $(1 - \alpha^{(k+1)}) / (1 - \alpha)$ = expected accepted tokens per round. Divide by $(kc + 1)$, where c is the draft model's cost relative to the target's, to get the actual speedup.
- Blindly increasing k stops paying off past a point. The lever that actually matters is α , the draft model's acceptance rate, not how far ahead it's allowed to guess.
- Discussion: given a fixed compute budget, would you invest it in a better-aligned draft model (raising α) or a longer speculation window (raising k)? What would change your answer?

Real-World Results, 2023–2026

- Rows 1-3 are wall-clock speedups from the last slide's formula. LK Losses measures acceptance-length gains; SSD measures speedup over the strongest prior speculative baseline, not over plain decoding.

Method	Venue	Speedup	Notes
Leviathan et al.	ICML 2023	2–3x	T5-XXL; exact output distribution preserved
EAGLE	ICML 2024	3.5x	LLaMA2-Chat 70B; throughput roughly doubled
Medusa	ICML 2024	2.8x	Medusa-2 configuration, no separate draft model
LK Losses	ICML 2026	8–10%↑	Directly optimizes α instead of KL divergence, 8B–685B params
Speculative Speculative Decoding	ICLR 2026	~30%↑	Parallelizes the drafting step itself with verification

Common Misconceptions

- “It's lossy.” False: accept/reject with residual resampling guarantees an output distribution identical to the target model alone.
- “Acceptance is a score match.” False: it's probabilistic, accept with probability $\min(1, q/p)$, not a direct score comparison.
- “More speculation always helps.” False: rejections accumulate; past a point, a longer speculation window wastes compute instead of saving it.

Learning Objectives

- Remember: state the two-model draft/target structure and the role of the acceptance ratio $q(x)/p(x)$.
- Understand: explain why parallel verification is cheap, and why residual resampling is required for correctness.
- Apply: implement the accept/reject and residual-sampling steps from a written specification.
- Analyze: decompose an observed speedup into a draft-quality axis (α) and a speculation-length axis (k).
- Evaluate: argue, given a fixed compute budget, whether draft quality or speculation depth yields the better return.
- Create: design a tree-based extension that verifies multiple candidate continuations per target call, as in EAGLE-2 and Medusa.